

PostgreSQL Database Exam — Set A

Total points 50/50 ?

Topics: PostgreSQL Intro, Install, Get Started, pgAdmin 4, CREATE TABLE, INSERT INTO, Fetch Data, ADD COLUMN, UPDATE, ALTER COLUMN, DROP COLUMN, DELETE, DROP TABLE, Demo Database, Operators, SELECT, SELECT DISTINCT, WHERE, ORDER BY, LIMIT

- Total questions: 50 (1 mark each, total 50 marks)
- All questions are compulsory.
- After you submit, click "View score" to see the correct answers and explanations.

0 of 0 points

Student Full Name *

MD MOKADDES HOSSAIN

Email *

monarulislam443@gmail.com

Set A — Questions

50 of 50 points

Choose ONE correct answer for each question.



✓ Customer #15 has moved. Their city must become 'Dhaka' and zip must become '1207'. Which query updates both columns correctly? *1/1

- ☒ UPDATE customers SET city = 'Dhaka', zip = '1207' WHERE customer_id = 15; ✓
- ☐ UPDATE customers SET city = 'Dhaka' WHERE customer_id = 15, zip = '1207';
- ☐ UPDATE customers SET city = 'Dhaka' AND zip = '1207' WHERE customer_id = 15;
- ☐ UPDATE customers SET (city, zip) WHERE customer_id = 15;

Feedback

✓ Correct!

✓ Correct answer: `UPDATE customers SET city = 'Dhaka', zip = '1207' WHERE customer_id = 15;`

Multiple columns in SET are separated by commas.

Why the other options are wrong:

✗ `UPDATE customers SET city = 'Dhaka' AND zip = '1207' WHERE customer_id = 15;`
→ This is read as `city = ('Dhaka' AND zip = '1207')`, a boolean expression, so it errors. Columns must be separated by commas, not AND.

✗ `UPDATE customers SET city = 'Dhaka' WHERE customer_id = 15, zip = '1207';`
→ SET assignments must come before WHERE. A comma inside WHERE like this is a syntax error.

✗ `UPDATE customers SET (city, zip) WHERE customer_id = 15;`
→ No values are given for the columns, so this is a syntax error.



✓ Look at the query below. Which rows will it return? *

1/1

```
SELECT * FROM products  
WHERE category = 'Toys' OR category = 'Books' AND price < 100;
```

- ☐ An error, because AND and OR cannot be mixed
- ☐ Only Books that cost less than 100
- ☒ All Toys (any price) plus Books that cost less than 100
- ☐ Toys and Books that both cost less than 100



Feedback

✓ Correct!

✓ Correct answer: All Toys (any price) plus Books that cost less than 100
AND is evaluated before OR, so this means Toys OR (Books AND price < 100). Use parentheses to change it.

Why the other options are wrong:

✗ Toys and Books that both cost less than 100

→ That needs parentheses: (category = 'Toys' OR category = 'Books') AND price < 100.

✗ Only Books that cost less than 100

→ The Toys condition stands alone on the left side of OR, so all Toys are included too.

✗ An error, because AND and OR cannot be mixed

→ AND and OR can be combined freely. You just need to know their precedence.



✓ Which statement correctly inserts one row into the table cars (brand, model, year)? *1/1

- ☐ ADD INTO cars (brand, model, year) VALUES ('Ford', 'Mustang', 1964);
- ☒ INSERT INTO cars (brand, model, year) VALUES ('Ford', 'Mustang', 1964); ✓
- ☐ INSERT INTO cars SET brand = 'Ford', model = 'Mustang', year = 1964;
- ☐ INSERT INTO cars (brand, model, year) VALUES ("Ford", "Mustang", 1964);

Feedback

✓ Correct!

✓ Correct answer: *INSERT INTO cars (brand, model, year) VALUES ('Ford', 'Mustang', 1964);*
This is the standard form: *INSERT INTO table (columns) VALUES (values)*. Text values use single quotes.

Why the other options are wrong:

X INSERT INTO cars (brand, model, year) VALUES ("Ford", "Mustang", 1964);
→ In PostgreSQL, double quotes mean identifiers (column or table names), so this fails with: column "Ford" does not exist.

X ADD INTO cars (brand, model, year) VALUES ('Ford', 'Mustang', 1964);
→ *ADD INTO* is not an SQL command.

X INSERT INTO cars SET brand = 'Ford', model = 'Mustang', year = 1964;
→ *INSERT ... SET* is MySQL-only syntax and gives an error in PostgreSQL.



✓ The marketing team needs a list of the different countries that customers *1/1 come from, with each country shown only once. Which query is correct?

- ☐ SELECT UNIQUE country FROM customers;
- ☒ SELECT DISTINCT country FROM customers;
- ☐ SELECT country FROM customers ORDER BY country;
- ☐ SELECT country FROM customers;



Feedback

✓ Correct!

✓ Correct answer: *SELECT DISTINCT country FROM customers;*
DISTINCT returns each country value only once.

Why the other options are wrong:

✗ *SELECT country FROM customers;*

→ This returns one row per customer, so countries repeat.

✗ *SELECT UNIQUE country FROM customers;*

→ *SELECT UNIQUE* is not valid in PostgreSQL. The keyword is *DISTINCT*.

✗ *SELECT country FROM customers ORDER BY country;*

→ This only sorts the list. Duplicates remain.



✓ Which statement removes the column "color" from the cars table? *

1/1

- ☐ DROP COLUMN color FROM cars;
- ☐ ALTER TABLE cars REMOVE color;
- ☐ DELETE COLUMN color FROM cars;
- ☒ ALTER TABLE cars DROP COLUMN color;



Feedback

✓ Correct!

✓ Correct answer: `ALTER TABLE cars DROP COLUMN color;`
Columns are removed with `ALTER TABLE ... DROP COLUMN column_name`.

Why the other options are wrong:

✗ `DELETE COLUMN color FROM cars;`

→ `DELETE` removes rows, not columns. "`DELETE COLUMN`" is not valid SQL.

✗ `ALTER TABLE cars REMOVE color;`

→ `REMOVE` is not a valid `ALTER TABLE` action in PostgreSQL.

✗ `DROP COLUMN color FROM cars;`

→ `DROP COLUMN` must be used inside `ALTER TABLE`.



- ✓ The customers table contains the (city, country) values shown below. *1/1
How many rows does this query return?
`SELECT DISTINCT city, country FROM customers;`

(Dhaka, Bangladesh)
(Dhaka, Bangladesh)
(Chattogram, Bangladesh)
(London, UK)

☒ 3

☐ 4

☐ 1

☐ 2



Feedback

✓ Correct!

✓ Correct answer: 3

With multiple columns, *DISTINCT* removes rows where the *COMBINATION* repeats. Only (Dhaka, Bangladesh) is duplicated, leaving 3 unique rows.

Why the other options are wrong:

✗ 2

→ 2 would be the result of *SELECT DISTINCT country*. Here the city is also considered.

✗ 4

→ 4 is the count without *DISTINCT*. The duplicate Dhaka row is removed.

✗ 1

→ There are clearly several different combinations.



✓ Which SQL command is used to modify the DATA in existing rows of a table? *1/1

- ☐ INSERT
- ☐ ALTER
- ☐ MODIFY
- ☒ UPDATE



Feedback

✓ Correct!

✓ Correct answer: *UPDATE*

UPDATE table SET column = value WHERE condition; changes values in existing rows.

Why the other options are wrong:

X ALTER

→ ALTER TABLE changes the table *STRUCTURE* (columns, types), not the row data.

X MODIFY

→ MODIFY is not a PostgreSQL command. It is part of MySQL's ALTER TABLE syntax.

X INSERT

→ INSERT adds *NEW* rows. It does not change existing ones.



✓ The "logs" table is full of old records. You must remove ALL rows but keep the table (columns, constraints) so new logs can be stored. Which statement should you use? *1/1

- ☐ ALTER TABLE logs DROP ALL ROWS;
- ☐ DELETE * FROM logs;
- ☐ DROP TABLE logs;
- ☒ DELETE FROM logs;



Feedback

✓ Correct!

✓ Correct answer: `DELETE FROM logs;`
`DELETE` without `WHERE` removes every row but keeps the table structure.

Why the other options are wrong:

✗ `DROP TABLE logs;`
→ `DROP TABLE` removes the table itself, so new logs would have nowhere to go.

✗ `DELETE * FROM logs;`
→ "`DELETE *`" is not valid PostgreSQL syntax and gives an error.

✗ `ALTER TABLE logs DROP ALL ROWS;`
→ `ALTER TABLE` changes structure and has no "`DROP ALL ROWS`" action.



✓ What is pgAdmin 4? *

1/1

- ☐ A command-line backup utility
- ☐ A programming language for writing stored procedures
- ☒ A graphical (GUI) tool to manage and query PostgreSQL databases ✓
- ☐ A separate database engine that replaces PostgreSQL

Feedback

✓ Correct!

✓ Correct answer: A graphical (GUI) tool to manage and query PostgreSQL databases
pgAdmin 4 is the official open-source GUI for PostgreSQL. You can browse objects, run queries with the Query Tool, and manage servers.

Why the other options are wrong:

✗ A separate database engine that replaces PostgreSQL

→ pgAdmin stores no data itself. It is a client that connects to a PostgreSQL server.

✗ A command-line backup utility

→ That describes pg_dump. pgAdmin is graphical.

✗ A programming language for writing stored procedures

→ Procedures are written in languages such as PL/pgSQL. pgAdmin is only a management tool.



✓ During installation, what is the name of the default superuser account that PostgreSQL creates? *1/1

- ☐ admin
- ☐ sa
- ☒ postgres
- ☐ root



Feedback

✓ Correct!

✓ Correct answer: postgres

The installer creates a superuser role called "postgres" and asks you to set its password.

Why the other options are wrong:

✗ root

→ "root" is the Linux superuser and MySQL's admin account, not PostgreSQL's.

✗ admin

→ PostgreSQL does not create an account named "admin" by default.

✗ sa

→ "sa" (system administrator) is the default admin login of Microsoft SQL Server.



✓ Look at the table and the INSERT statement below. What will be stored in *1/1 the new row?

```
CREATE TABLE students (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(50),  
  city VARCHAR(50)  
);
```

```
INSERT INTO students (name) VALUES ('Rahim');
```

- ☐ id is stored as NULL
- ☐ The INSERT fails because city is missing
- ☐ city is stored as an empty string "
- ☒ id gets the next auto number, name = 'Rahim', city = NULL



Feedback

✓ Correct!

✓ Correct answer: id gets the next auto number, name = 'Rahim', city = NULL
SERIAL fills id automatically. city was not given and has no DEFAULT or NOT NULL, so it becomes NULL.

Why the other options are wrong:

✗ The INSERT fails because city is missing
→ Columns may be left out unless they are NOT NULL without a default. city has no such constraint.

✗ city is stored as an empty string "
→ An omitted column gets its DEFAULT, which is NULL here. An empty string is a different value.

✗ id is stored as NULL
→ A SERIAL PRIMARY KEY is never NULL. The sequence generates the next number.



✓ What is the result of `SELECT 7 / 2;` in PostgreSQL? *

1/1

☐ An error

☒ 3



☐ 3.5

☐ 4

Feedback

✓ Correct!

✓ Correct answer: 3

Both operands are integers, so PostgreSQL does integer division and drops the decimal part. Use `7.0 / 2` to get 3.5.

Why the other options are wrong:

✗ 3.5

→ You get 3.5 only if one operand is decimal, for example `7.0 / 2` or `7::numeric / 2`.

✗ 4

→ Integer division truncates. It does not round up.

✗ An error

→ Dividing integers is valid. Only dividing by zero gives an error.



✓ You have just installed PostgreSQL on Windows. You want the command-line terminal that ships with PostgreSQL so you can connect to the server and type SQL statements. Which tool should you open? *1/1

- ☐ pgAdmin 4
- ☐ pg_dump
- ☐ Windows Notepad
- ☒ SQL Shell (psql)



Feedback

✓ Correct!

✓ Correct answer: SQL Shell (psql)

psql is PostgreSQL's interactive command-line terminal. It is installed along with the server, and you type SQL directly into it.

Why the other options are wrong:

✗ pgAdmin 4

→ pgAdmin 4 is also bundled, but it is a graphical (GUI) tool, not a command-line terminal.

✗ pg_dump

→ pg_dump is a backup utility that exports a database. It is not an interactive SQL terminal.

✗ Windows Notepad

→ Notepad is only a text editor. It cannot connect to a database server or run SQL.



✓ You need to add a column "status" to the users table. Every existing user *1/1
AND every future user should automatically have the value 'active' unless
another value is given. Which statement does this in one step?

- ☐ ALTER TABLE users ADD COLUMN status VARCHAR(10);
- ☒ ALTER TABLE users ADD COLUMN status VARCHAR(10) DEFAULT 'active'; ✓
- ☐ ALTER TABLE users ADD COLUMN status = 'active';
- ☐ UPDATE users SET status = 'active';

Feedback

✓ Correct!

✓ Correct answer: `ALTER TABLE users ADD COLUMN status VARCHAR(10) DEFAULT 'active';`

With `DEFAULT`, PostgreSQL fills existing rows with 'active' and also uses it for future `INSERTs` that leave out status.

Why the other options are wrong:

✗ `ALTER TABLE users ADD COLUMN status VARCHAR(10);`

→ This adds the column, but existing rows get `NULL` and future rows get `NULL` too.

✗ `UPDATE users SET status = 'active';`

→ The column does not exist yet, so this fails. Even if it existed, it would not affect future rows.

✗ `ALTER TABLE users ADD COLUMN status = 'active';`

→ `"= 'active'"` is not valid in a column definition. The syntax is `DEFAULT 'active'`.



✓ An e-commerce homepage must show the 3 MOST expensive products. *1/1
Which query is correct in PostgreSQL?

- ☒ SELECT * FROM products ORDER BY price DESC LIMIT 3; ✓
- ☐ SELECT * FROM products ORDER BY price ASC LIMIT 3;
- ☐ SELECT TOP 3 * FROM products ORDER BY price DESC;
- ☐ SELECT * FROM products LIMIT 3;

Feedback

✓ Correct!

✓ Correct answer: *SELECT * FROM products ORDER BY price DESC LIMIT 3;*
Sort from highest to lowest price, then take the first 3 rows.

Why the other options are wrong:

✗ *SELECT * FROM products LIMIT 3;*

→ Without *ORDER BY*, you get 3 arbitrary rows, not the most expensive.

✗ *SELECT * FROM products ORDER BY price ASC LIMIT 3;*

→ *ASC* puts the cheapest first, so this returns the 3 cheapest products.

✗ *SELECT TOP 3 * FROM products ORDER BY price DESC;*

→ *TOP* is SQL Server syntax. PostgreSQL uses *LIMIT*.



✓ Your setup script must remove the table "temp_data" but should NOT stop with an error if the table does not exist. Which statement should be used? *1/1

- ☒ DROP TABLE IF EXISTS temp_data; ✓
- ☐ DROP TABLE temp_data IF EXIST;
- ☐ DELETE TABLE IF EXISTS temp_data;
- ☐ DROP IF EXISTS TABLE temp_data;

Feedback

✓ Correct!

✓ Correct answer: *DROP TABLE IF EXISTS temp_data;*
IF EXISTS makes PostgreSQL skip the drop with only a notice when the table is missing, so the script continues.

Why the other options are wrong:

✗ DROP TABLE temp_data IF EXIST;

→ IF EXISTS must come before the table name, and "EXIST" is misspelled.

✗ DELETE TABLE IF EXISTS temp_data;

→ DELETE TABLE is not a command. DELETE removes rows.

✗ DROP IF EXISTS TABLE temp_data;

→ The keyword order is wrong. The correct order is DROP TABLE IF EXISTS name.



✓ Which operator joins (concatenates) two text values in PostgreSQL, for example first_name and last_name? *1/1

- ☐ +
- ☒ ||
- ☐ &&
- ☐ &



Feedback

✓ Correct!

✓ Correct answer: ||

'Rahim' || ' ' || 'Uddin' gives 'Rahim Uddin'. || is the SQL string concatenation operator.

Why the other options are wrong:

X +

→ + is for numbers. text + text gives the error "operator does not exist".

X &

→ & is the bitwise AND operator for integers.

X &&

→ && is the overlap operator for arrays and ranges, not concatenation.



✓ The column "phone" in the contacts table is VARCHAR(10), but new international numbers need up to 15 characters. How should you fix this WITHOUT losing existing data? *1/1

- ☐ UPDATE contacts SET phone = VARCHAR(15);
- ☐ ALTER TABLE contacts ADD COLUMN phone VARCHAR(15);
- ☒ ALTER TABLE contacts ALTER COLUMN phone TYPE VARCHAR(15); ✓
- ☐ DROP TABLE contacts; then create it again with VARCHAR(15)

Feedback

✓ Correct!

✓ Correct answer: `ALTER TABLE contacts ALTER COLUMN phone TYPE VARCHAR(15);`
Changing the type to a longer VARCHAR keeps all existing values and allows longer ones.

Why the other options are wrong:

✗ `DROP TABLE contacts; then create it again with VARCHAR(15)`

→ Dropping the table deletes ALL existing data. It is unnecessary and destructive.

✗ `UPDATE contacts SET phone = VARCHAR(15);`

→ UPDATE changes values, not data types. This statement is invalid.

✗ `ALTER TABLE contacts ADD COLUMN phone VARCHAR(15);`

→ A column named phone already exists, so this fails with: column "phone" already exists.



✓ Which WHERE clause correctly selects customers whose country is Mexico? *1/1

- ☒ WHERE country = 'Mexico' ✓
- ☐ WHERE country == 'Mexico'
- ☐ WHERE country IS 'Mexico'
- ☐ WHERE country = "Mexico"

Feedback

✓ Correct!

✓ Correct answer: *WHERE country = 'Mexico'*
Text values go in single quotes, and equality is a single =.

Why the other options are wrong:

✗ *WHERE country = "Mexico"*
→ Double quotes mean an identifier (column name) in PostgreSQL, so this gives: column "Mexico" does not exist.

✗ *WHERE country == 'Mexico'*
→ == is not an SQL operator.

✗ *WHERE country IS 'Mexico'*
→ IS is used with NULL, TRUE or FALSE, not for comparing text values.



- ✓ While loading the demo database, you run the INSERT below. *1/1
orders.customer_id is a FOREIGN KEY to customers, and no customer with ID 999 exists. What happens?

```
INSERT INTO orders (order_id, customer_id, order_date)
VALUES (5001, 999, '2024-05-10');
```

- ☒ The INSERT fails with a foreign key constraint violation ✓
- ☐ The order is inserted with customer_id set to NULL
- ☐ The order is inserted normally
- ☐ A new customer with ID 999 is created automatically

Feedback

✓ Correct!

✓ *Correct answer: The INSERT fails with a foreign key constraint violation*
PostgreSQL checks the referenced table. Since customer 999 does not exist, it rejects the row.

Why the other options are wrong:

✗ *The order is inserted with customer_id set to NULL*
→ PostgreSQL never silently changes your values. It raises an error.

✗ *A new customer with ID 999 is created automatically*
→ Foreign keys do not create parent rows.

✗ *The order is inserted normally*
→ Allowing this is exactly what a foreign key prevents.



✓ If you write `ORDER BY price` without `ASC` or `DESC`, how are the rows sorted?

*1/1

- ☐ Descending (largest to smallest)
- ☐ In insertion order
- ☒ Ascending (smallest to largest)
- ☐ In random order



Feedback

✓ Correct!

✓ Correct answer: Ascending (smallest to largest)
ASC is the default sort direction.

Why the other options are wrong:

✗ Descending (largest to smallest)
→ DESC must be written explicitly.

✗ In random order
→ Random order needs `ORDER BY random()`.

✗ In insertion order
→ Without `ORDER BY`, row order is not guaranteed. With `ORDER BY price`, rows are sorted by price.



✓ Which statement adds a new column "color" to the existing cars table? * 1/1

- ☐ UPDATE TABLE cars ADD color VARCHAR(255);
- ☒ ALTER TABLE cars ADD COLUMN color VARCHAR(255);
- ☐ ALTER cars ADD COLUMN color VARCHAR(255);
- ☐ INSERT COLUMN color INTO cars;



Feedback

✓ Correct!

✓ Correct answer: `ALTER TABLE cars ADD COLUMN color VARCHAR(255);`
Table structure is changed with `ALTER TABLE`. `ADD COLUMN` name datatype adds the new column.

Why the other options are wrong:

✗ `UPDATE TABLE cars ADD color VARCHAR(255);`
→ `UPDATE` changes data in rows. It cannot change table structure, and "`UPDATE TABLE`" is not valid syntax.

✗ `ALTER cars ADD COLUMN color VARCHAR(255);`
→ The `TABLE` keyword is missing, which is a syntax error.

✗ `INSERT COLUMN color INTO cars;`
→ `INSERT` adds rows, not columns. This is not valid SQL.



- ✓ The students table already contains 500 rows. The admin runs the statement below. What value will the existing 500 rows have in the new column? *1/1

ALTER TABLE students ADD COLUMN email VARCHAR(100);

- ☐ The statement fails because the table already has data
- ☐ All 500 rows are deleted
- ☐ An empty string "
- ☒ NULL ✓

Feedback

✓ Correct!

✓ Correct answer: NULL

A newly added column without a DEFAULT is filled with NULL for all existing rows.

Why the other options are wrong:

✗ The statement fails because the table already has data

→ Columns can be added to tables that already contain data.

✗ All 500 rows are deleted

→ ADD COLUMN never deletes rows.

✗ An empty string "

→ An empty string is a real value. PostgreSQL uses NULL when no default is given.



✓ A library needs a "books" table in which every book automatically gets a unique, auto-incrementing ID. Which statement is correct in PostgreSQL? *1/1

- ☐ CREATE books (book_id SERIAL PRIMARY KEY, title VARCHAR(200));
- ☒ CREATE TABLE books (book_id SERIAL PRIMARY KEY, title VARCHAR(200) NOT NULL); ✓
- ☐ CREATE TABLE books (book_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(200));
- ☐ CREATE TABLE books (book_id INT, title VARCHAR(200));

Feedback

✓ Correct!

✓ Correct answer: `CREATE TABLE books (book_id SERIAL PRIMARY KEY, title VARCHAR(200) NOT NULL);`
`SERIAL` creates an auto-incrementing integer (via a sequence), and `PRIMARY KEY` guarantees each ID is unique and not `NULL`.

Why the other options are wrong:

✗ `CREATE TABLE books (book_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(200));`

→ `AUTO_INCREMENT` is MySQL syntax. PostgreSQL gives a syntax error. Use `SERIAL` (or `GENERATED ... AS IDENTITY`).

✗ `CREATE TABLE books (book_id INT, title VARCHAR(200));`

→ A plain `INT` is not auto-generated and has no uniqueness rule, so IDs must be typed by hand and could repeat.

✗ `CREATE books (book_id SERIAL PRIMARY KEY, title VARCHAR(200));`

→ The `TABLE` keyword is missing, which is a syntax error.



✓ What is the purpose of SELECT DISTINCT? *

1/1

- ☐ To count the number of rows
- ☐ To permanently delete duplicate rows from the table
- ☐ To sort the result alphabetically
- ☒ To return only unique (non-duplicate) values or rows in the result



Feedback

✓ Correct!

✓ Correct answer: To return only unique (non-duplicate) values or rows in the result
DISTINCT removes duplicate rows from the query RESULT only.

Why the other options are wrong:

*✗ To sort the result alphabetically
→ Sorting is done with ORDER BY.*

*✗ To count the number of rows
→ Counting is done with COUNT().*

*✗ To permanently delete duplicate rows from the table
→ DISTINCT affects only the output. The table data is not changed.*



✓ What is the correct order of clauses in a SELECT statement? *

1/1

- ☐ SELECT → FROM → ORDER BY → WHERE → LIMIT
- ☐ FROM → SELECT → WHERE → LIMIT → ORDER BY
- ☐ SELECT → WHERE → FROM → ORDER BY → LIMIT
- ☒ SELECT → FROM → WHERE → ORDER BY → LIMIT



Feedback

✓ Correct!

✓ Correct answer: *SELECT → FROM → WHERE → ORDER BY → LIMIT*
This is the required written order. Placing clauses differently gives a syntax error.

Why the other options are wrong:

~~X~~ *SELECT → WHERE → FROM → ORDER BY → LIMIT*
→ FROM must come before WHERE.

~~X~~ *FROM → SELECT → WHERE → LIMIT → ORDER BY*
→ A query starts with SELECT, and LIMIT comes after ORDER BY.

~~X~~ *SELECT → FROM → ORDER BY → WHERE → LIMIT*
→ WHERE must come before ORDER BY.



✓ In pgAdmin 4 you have selected the database "school" and want to write and execute your own SQL statement against it. Which pgAdmin feature should you open? *1/1

- ☐ Dashboard
- ☐ Properties dialog
- ☒ Query Tool
- ☐ Backup dialog



Feedback

✓ Correct!

✓ Correct answer: Query Tool

The Query Tool (Tools → Query Tool) opens an SQL editor connected to the selected database, where you can write and execute queries.

Why the other options are wrong:

✗ Dashboard

→ The Dashboard shows server activity and statistics. You cannot write SQL there.

✗ Properties dialog

→ Properties shows or edits settings of an object (owner, comment, etc.). It does not execute custom SQL.

✗ Backup dialog

→ Backup exports the database to a file. It is not used to run your own queries.



✓ By default, which TCP port does a PostgreSQL server listen on? *

1/1

☐ 1433

☒ 5432



☐ 8080

☐ 3306

Feedback

✓ Correct!

✓ Correct answer: 5432

5432 is PostgreSQL's default port, suggested during installation and used by psql/pgAdmin when you connect.

Why the other options are wrong:

✗ 3306

→ 3306 is MySQL's default port.

✗ 1433

→ 1433 is Microsoft SQL Server's default port.

✗ 8080

→ 8080 is commonly used for web/HTTP servers, not PostgreSQL.



- ✓ A shop owner wants to add three products (Pen 10 Tk, Pencil 5 Tk, Eraser 3 Tk) to the table products(name, price) using a SINGLE statement. Which one works? *1/1

- ☐ INSERT INTO products (name, price) VALUES [('Pen', 10), ('Pencil', 5), ('Eraser', 3)];
- ☐ INSERT INTO products (name, price) VALUES ('Pen', 10); ('Pencil', 5); ('Eraser', 3);
- ☐ INSERT INTO products (name, price) VALUES ('Pen', 'Pencil', 'Eraser'), (10, 5, 3);
- ☒ INSERT INTO products (name, price) VALUES ('Pen', 10), ('Pencil', 5), ('Eraser', 3); ✓

Feedback

✓ Correct!

✓ Correct answer: `INSERT INTO products (name, price) VALUES ('Pen', 10), ('Pencil', 5), ('Eraser', 3);`

PostgreSQL lets you insert several rows at once by separating each row's value list with a comma.

Why the other options are wrong:

✗ `INSERT INTO products (name, price) VALUES ('Pen', 10); ('Pencil', 5); ('Eraser', 3);`

→ The semicolon ends the statement after the first row. `"('Pencil', 5);"` is then a separate, invalid statement, which gives a syntax error.

✗ `INSERT INTO products (name, price) VALUES [('Pen', 10), ('Pencil', 5), ('Eraser', 3)];`

→ Square brackets are not valid around a VALUES list.

✗ `INSERT INTO products (name, price) VALUES ('Pen', 'Pencil', 'Eraser'), (10, 5, 3);`

→ Each parenthesis group is one ROW. Here each row has 3 values for 2 columns, so this is an error.



✓ What does the NOT NULL constraint on a column guarantee? *

1/1

- ☒ The column must always contain a value. NULL is not allowed. ✓
- ☐ Every value in the column must be unique
- ☐ The column cannot contain the number 0
- ☐ The column automatically gets a default value

Feedback

✓ Correct!

✓ Correct answer: The column must always contain a value. NULL is not allowed.
NOT NULL rejects any INSERT or UPDATE that would leave the column empty (NULL).

Why the other options are wrong:

✗ Every value in the column must be unique

→ That is the UNIQUE constraint. NOT NULL allows duplicate values.

✗ The column cannot contain the number 0

→ 0 is a real value, not NULL, so it is allowed. A CHECK constraint would be needed to block 0.

✗ The column automatically gets a default value

→ That is the DEFAULT clause. NOT NULL only forbids missing values.



✓ Which language is used to create, read, update and delete data in PostgreSQL?

*1/1

- ☐ HTML
- ☐ MongoDB Query Language (MQL)
- ☐ CSS
- ☒ SQL (Structured Query Language)



Feedback

✓ Correct!

✓ Correct answer: SQL (Structured Query Language)

PostgreSQL is an SQL database. All data operations (*SELECT, INSERT, UPDATE, DELETE, CREATE ...*) are written in SQL.

Why the other options are wrong:

✗ HTML

→ HTML is a markup language for web pages. It cannot talk to a database.

✗ MongoDB Query Language (MQL)

→ MQL is used by MongoDB, a NoSQL database, not by PostgreSQL.

✗ CSS

→ CSS styles web pages and has nothing to do with querying a database.



✓ Which operator means "not equal to" in PostgreSQL? *

1/1

☒ <>



☐ ==

☐ =!

☐ ><

Feedback

✓ Correct!

✓ Correct answer: <>

<> is the standard SQL "not equal" operator. PostgreSQL also accepts != as a synonym.

Why the other options are wrong:

X ==

→ == is not an SQL operator. Equality in SQL is a single =.

X =!

→ != is not a valid comparison operator.

X ><

→ >< is not a valid comparison operator.



- ✓ Some rows in the users table currently have NULL in the email column. *1/1
The admin runs the statement below. What happens?

ALTER TABLE users ALTER COLUMN email SET NOT NULL;

- ☐ NULL emails are converted to empty strings
- ☒ It fails with an error because existing rows contain NULL values ✓
- ☐ Rows with NULL email are deleted automatically
- ☐ The rule is applied only to new rows; old NULLs are ignored

Feedback

✓ Correct!

✓ Correct answer: It fails with an error because existing rows contain NULL values
PostgreSQL checks all existing rows. If any contain NULL, it reports "column email contains null values". Fill or remove those NULLs first.

Why the other options are wrong:

✗ Rows with NULL email are deleted automatically
→ ALTER COLUMN never deletes rows.

✗ NULL emails are converted to empty strings
→ PostgreSQL does not silently change data.

✗ The rule is applied only to new rows; old NULLs are ignored
→ A NOT NULL constraint must hold for ALL rows, old and new.



✓ What is PostgreSQL? *

1/1

- ☒ An open-source object-relational database management system (ORDBMS) ✓
- ☐ A NoSQL document-only database
- ☐ A front-end language used to design web pages
- ☐ A paid, closed-source database owned by Oracle

Feedback

✓ Correct!

✓ Correct answer: An open-source object-relational database management system (ORDBMS)

PostgreSQL is a free, open-source, object-relational DBMS. It stores data in tables and is queried using SQL.

Why the other options are wrong:

✗ A NoSQL document-only database

→ PostgreSQL can store JSON, but it is fundamentally a relational (SQL) database, not a document-only NoSQL store.

✗ A front-end language used to design web pages

→ It is a database server, not a web page language. HTML, CSS and JavaScript are used for web pages.

✗ A paid, closed-source database owned by Oracle

→ PostgreSQL is open source and community-developed. Oracle Database and MySQL are the products owned by Oracle.



✓ HR wants the employee list with the highest salary first. If two employees *1/1 have the same salary, they should appear alphabetically by name. Which ORDER BY is correct?

- ☒ ORDER BY salary DESC, name ASC ✓
- ☐ ORDER BY salary, name DESC
- ☐ ORDER BY DESC salary, name
- ☐ ORDER BY name ASC, salary DESC

Feedback

✓ Correct!

✓ Correct answer: ORDER BY salary DESC, name ASC

The first column sorts salaries from high to low. Ties are broken by the second column, name A→Z.

Why the other options are wrong:

✗ ORDER BY salary, name DESC

→ salary with no direction is ASC (lowest first), and names would be Z→A.

✗ ORDER BY name ASC, salary DESC

→ This sorts mainly by name, so salary only breaks ties between identical names.

✗ ORDER BY DESC salary, name

→ The direction keyword must come AFTER the column, so this is a syntax error.



✓ After running `ALTER TABLE students DROP COLUMN address;` what happens to the data in that column? *1/1

- ☐ The column is hidden and can be restored with `UNDROP`
- ☒ The column and all its values are permanently removed; other columns stay unchanged ✓
- ☐ The entire students table is deleted
- ☐ The values become `NULL` but the column remains

Feedback

✓ Correct!

✓ Correct answer: The column and all its values are permanently removed; other columns stay unchanged

`DROP COLUMN` deletes the column definition and its data from every row. The remaining columns and rows are not affected.

Why the other options are wrong:

✗ The values become `NULL` but the column remains

→ That would be `UPDATE students SET address = NULL`. `DROP` removes the column itself.

✗ The entire students table is deleted

→ Only `DROP TABLE` removes the whole table.

✗ The column is hidden and can be restored with `UNDROP`

→ There is no `UNDROP` command. Without a backup or an open transaction, the data is gone.



✓ Which data type is most suitable for storing mobile numbers such as +8801712345678 (keeping the "+" and any leading zero)? *1/1

☒ VARCHAR(20) ✓

☐ DATE

☐ BOOLEAN

☐ INT

Feedback

✓ Correct!

✓ Correct answer: VARCHAR(20)

A phone number is an identifier, not a quantity. VARCHAR keeps the "+", leading zeros and formatting exactly as entered.

Why the other options are wrong:

X INT

→ INT cannot store "+" or leading zeros, and 8801712345678 exceeds INT's maximum (2,147,483,647).

X BOOLEAN

→ BOOLEAN stores only true/false.

X DATE

→ DATE stores calendar dates, not phone numbers.



✓ Which query returns ALL columns and ALL rows of the customers table? * 1/1

- ☐ FETCH ALL FROM customers;
- ☒ SELECT * FROM customers;
- ☐ GET * FROM customers;
- ☐ SELECT ALL customers;



Feedback

✓ Correct!

✓ Correct answer: *SELECT * FROM customers;*
** means "all columns". Without a WHERE clause, every row is returned.*

Why the other options are wrong:

X *SELECT ALL customers;*

→ *This is invalid. SELECT needs a column list and FROM table_name.*

X *GET * FROM customers;*

→ *GET is not an SQL command.*

X *FETCH ALL FROM customers;*

→ *FETCH reads from a CURSOR, not a table. This fails with: cursor "customers" does not exist.*



✓ A website shows 10 products per page, sorted by product_id. Which query returns the products for PAGE 3? *1/1

- ☒ SELECT * FROM products ORDER BY product_id LIMIT 10 OFFSET 20; ✓
- ☐ SELECT * FROM products ORDER BY product_id LIMIT 10 OFFSET 30;
- ☐ SELECT * FROM products ORDER BY product_id LIMIT 30 OFFSET 10;
- ☐ SELECT * FROM products ORDER BY product_id LIMIT 3 OFFSET 10;

Feedback

✓ Correct!

✓ Correct answer: `SELECT * FROM products ORDER BY product_id LIMIT 10 OFFSET 20;`
Pages 1 and 2 cover 20 rows, so skip 20 (OFFSET 20) and take 10 (LIMIT 10).

Why the other options are wrong:

✗ `SELECT * FROM products ORDER BY product_id LIMIT 10 OFFSET 30;`
→ OFFSET 30 skips 3 pages, which returns page 4.

✗ `SELECT * FROM products ORDER BY product_id LIMIT 30 OFFSET 10;`
→ This returns 30 rows (pages 2 to 4). The numbers are swapped.

✗ `SELECT * FROM products ORDER BY product_id LIMIT 3 OFFSET 10;`
→ LIMIT is the number of rows per page (10), not the page number.



✓ The employees table has a manager_id column (INTEGER). The CEO has no manager, so their manager_id is NULL. Which query finds employees who have NO manager? *1/1

- ☐ SELECT * FROM employees WHERE manager_id = 0;
- ☐ SELECT * FROM employees WHERE manager_id = 'NULL';
- ☒ SELECT * FROM employees WHERE manager_id IS NULL;
- ☐ SELECT * FROM employees WHERE manager_id = NULL;



Feedback

✓ Correct!

✓ Correct answer: `SELECT * FROM employees WHERE manager_id IS NULL;`
NULL means "unknown", so it must be tested with `IS NULL`.

Why the other options are wrong:

✗ `SELECT * FROM employees WHERE manager_id = NULL;`
→ Any comparison with NULL using `=` gives UNKNOWN (not TRUE), so this returns 0 rows.

✗ `SELECT * FROM employees WHERE manager_id = 'NULL';`
→ 'NULL' is just text. On an INTEGER column, this gives an invalid input error.

✗ `SELECT * FROM employees WHERE manager_id = 0;`
→ 0 is a real number, not NULL, so the CEO's row will not match.



- ✓ In SQL Shell (psql), a student types the line below and presses Enter. *1/1
Nothing is returned, and the prompt changes from postgres=# to postgres-#. What is the reason?

postgres=# SELECT version()

- ☒ The statement is missing the ending semicolon (;), so psql is waiting for more input ✓
- ☐ SQL keywords must be written in lowercase
- ☐ version() is not a valid PostgreSQL function
- ☐ The PostgreSQL server has stopped

Feedback

✓ Correct!

✓ Correct answer: The statement is missing the ending semicolon (;), so psql is waiting for more input

psql sends a statement to the server only after it sees a semicolon. The "-#" prompt means a continuation line. Typing ; and pressing Enter runs the query.

Why the other options are wrong:

✗ The PostgreSQL server has stopped

→ If the server were down, psql would show a connection error, not a continuation prompt.

✗ version() is not a valid PostgreSQL function

→ version() is a built-in function that returns the PostgreSQL version. An invalid function would produce an error message.

✗ SQL keywords must be written in lowercase

→ SQL keywords are case-insensitive, so SELECT and select behave the same.



- ✓ A clerk wanted to change only employee #7's salary but ran the query below. What happens? *1/1

UPDATE employees SET salary = 50000;

- ☐ Only employee #7 is updated
- ☐ Only the first row's salary is changed
- ☐ PostgreSQL returns an error because WHERE is mandatory
- ☒ Every employee's salary becomes 50000 ✓

Feedback

✓ Correct!

✓ Correct answer: Every employee's salary becomes 50000

Without a WHERE clause, UPDATE changes ALL rows. Always add WHERE, for example WHERE emp_id = 7.

Why the other options are wrong:

✗ Only the first row's salary is changed

→ UPDATE has no "first row" behaviour. It applies to every row that matches, which here is all of them.

✗ PostgreSQL returns an error because WHERE is mandatory

→ WHERE is optional in UPDATE. That is why forgetting it is dangerous.

✗ Only employee #7 is updated

→ Nothing in the query refers to employee 7, so PostgreSQL cannot know that intention.



✓ What does DROP TABLE cars; do? *

1/1

- ☐ Renames the table to cars_old
- ☐ Deletes only the rows but keeps the empty table
- ☒ Permanently deletes the table's structure and all its data
- ☐ Deletes only the first row



Feedback

✓ Correct!

✓ Correct answer: Permanently deletes the table's structure and all its data
DROP TABLE removes the table definition, all rows, indexes and constraints.

Why the other options are wrong:

✗ Deletes only the rows but keeps the empty table
→ That is DELETE FROM cars; (or TRUNCATE).

✗ Deletes only the first row
→ DROP TABLE works on the entire table, not individual rows.

✗ Renames the table to cars_old
→ Renaming uses ALTER TABLE cars RENAME TO new_name.



✓ Which SQL statement is used to create a new table in PostgreSQL? * 1/1

- ☒ CREATE TABLE
- ☐ NEW TABLE
- ☐ MAKE TABLE
- ☐ INSERT TABLE



Feedback

✓ Correct!

✓ Correct answer: CREATE TABLE

CREATE TABLE table_name (column datatype, ...); defines a new table and its columns.

Why the other options are wrong:

✗ NEW TABLE

→ NEW TABLE is not an SQL command.

✗ MAKE TABLE

→ MAKE TABLE is not valid in PostgreSQL. It is a Microsoft Access query type.

✗ INSERT TABLE

→ INSERT adds rows to an existing table. It does not create tables.



✓ Which PostgreSQL statement changes the data type of the column "year" *1/1
in the cars table to VARCHAR(4)?

- ☐ ALTER TABLE cars MODIFY COLUMN year VARCHAR(4);
- ☒ ALTER TABLE cars ALTER COLUMN year TYPE VARCHAR(4); ✓
- ☐ ALTER TABLE cars CHANGE year VARCHAR(4);
- ☐ UPDATE cars SET year TYPE VARCHAR(4);

Feedback

✓ Correct!

✓ Correct answer: `ALTER TABLE cars ALTER COLUMN year TYPE VARCHAR(4);`
PostgreSQL syntax: `ALTER TABLE ... ALTER COLUMN column TYPE new_type.`

Why the other options are wrong:

✗ `ALTER TABLE cars MODIFY COLUMN year VARCHAR(4);`
→ `MODIFY COLUMN` is MySQL syntax and gives an error in PostgreSQL.

✗ `UPDATE cars SET year TYPE VARCHAR(4);`
→ `UPDATE` changes data, not column definitions.

✗ `ALTER TABLE cars CHANGE year VARCHAR(4);`
→ `CHANGE` is MySQL syntax and is not supported by PostgreSQL.



✓ Due to a new tax, the price of every product in the 'Electronics' category must increase by 10%. Which query is correct? *1/1

- ☐ UPDATE products SET price = price + 10% WHERE category = 'Electronics';
- ☒ UPDATE products SET price = price * 1.10 WHERE category = 'Electronics'; ✓
- ☐ UPDATE products SET price = 1.10 WHERE category = 'Electronics';
- ☐ UPDATE products price * 1.10 WHERE category = 'Electronics';

Feedback

✓ Correct!

✓ Correct answer: `UPDATE products SET price = price * 1.10 WHERE category = 'Electronics';`
price * 1.10 adds 10% to the current price, and WHERE limits the change to Electronics.

Why the other options are wrong:

✗ `UPDATE products SET price = price + 10% WHERE category = 'Electronics';`
→ "10%" is not a valid expression in SQL (% is the modulo operator), so this is a syntax error.

✗ `UPDATE products SET price = 1.10 WHERE category = 'Electronics';`
→ This sets every Electronics price to 1.10 Tk instead of increasing it.

✗ `UPDATE products price * 1.10 WHERE category = 'Electronics';`
→ The SET keyword and the "column =" part are missing, so this is a syntax error.



✓ A customer wants to see products priced from 100 Tk up to (but not including) 500 Tk. Which WHERE clause is correct? *1/1

- ☒ WHERE price >= 100 AND price < 500 ✓
- ☐ WHERE price >= 100 OR price < 500
- ☐ WHERE price >= 100 < 500
- ☐ WHERE price => 100 AND price < 500

Feedback

✓ Correct!

✓ Correct answer: WHERE price >= 100 AND price < 500
Both conditions must be true, so AND is used: at least 100 and below 500.

Why the other options are wrong:

✗ WHERE price >= 100 OR price < 500

→ With OR, almost every product matches (any price ≥ 100 OR any price < 500).

✗ WHERE price >= 100 < 500

→ Comparisons cannot be chained like this in SQL, so this gives an error.

✗ WHERE price => 100 AND price < 500

→ => is not the "greater than or equal" operator (that is >=), so this causes an error.



✓ An accountant wants to SEE each product's name and its price including 15% VAT, without changing any stored data. Which query should be used? *1/1

- ☒ SELECT name, price * 1.15 FROM products;
- ☐ UPDATE products SET price = price * 1.15;
- ☐ SELECT name, price + 15% FROM products;
- ☐ SELECT name, price FROM products * 1.15;



Feedback

✓ Correct!

✓ Correct answer: *SELECT name, price * 1.15 FROM products;*
SELECT can calculate expressions on the fly. The stored price is unchanged.

Why the other options are wrong:

✗ SELECT name, price + 15% FROM products;
→ "15%" is not valid in SQL (% is modulo), so this is a syntax error.

*✗ SELECT name, price FROM products * 1.15;*
→ You cannot multiply a table. The calculation belongs in the SELECT list.

*✗ UPDATE products SET price = price * 1.15;*
→ This permanently changes the stored prices, which is exactly what the accountant does NOT want.



✓ What does the statement `DELETE FROM cars WHERE brand = 'Volvo';` do? * 1/1

- ☐ Deletes the cars table
- ☐ Removes the brand column from the table
- ☐ Sets brand to NULL for Volvo cars
- ☒ Removes every row whose brand is 'Volvo'



Feedback

✓ Correct!

✓ *Correct answer: Removes every row whose brand is 'Volvo'*
DELETE removes the ROWS that match the WHERE condition. The table and other rows remain.

Why the other options are wrong:

X Removes the brand column from the table
→ Removing a column needs `ALTER TABLE ... DROP COLUMN`.

X Deletes the cars table
→ Deleting the table needs `DROP TABLE`.

X Sets brand to NULL for Volvo cars
→ That would be an `UPDATE`. `DELETE` removes the whole row.



✓ In the demo database, `orders.customer_id` must always refer to an existing `customers.customer_id`. Which constraint enforces this relationship? *1/1

- ☐ UNIQUE
- ☒ FOREIGN KEY
- ☐ PRIMARY KEY
- ☐ CHECK



Feedback

✓ Correct!

✓ Correct answer: *FOREIGN KEY*

A *FOREIGN KEY* (`orders.customer_id REFERENCES customers(customer_id)`) ensures every order points to a real customer.

Why the other options are wrong:

✗ *PRIMARY KEY*

→ A primary key uniquely identifies rows inside its own table. It does not link to another table.

✗ *UNIQUE*

→ *UNIQUE* only prevents duplicates in the column. One customer can have many orders, so it is also the wrong rule here.

✗ *CHECK*

→ *CHECK* validates a condition on the row's own values (for example `price > 0`). It cannot look up another table.

This content is neither created nor endorsed by Google. - [Terms of Service](#) - [Privacy Policy](#)

Does this form look suspicious? [Report](#)

Google Forms



